CSC 410: Big Data and Machine Learning.

Ashish Khanal

Fall 2025 Assignment 4

Abstract- This document is a write-up for CSC-410 Assignment 4. The goal for this assignment is to learn how language models and multimodal models work by using two datasets: the *Jokes dataset* and the *Meme dataset*. In the first part, I trained a Transformer-based language model from scratch, start to finish. I developed/ created my own tokenizer using Byte-Pair Encoding (BPE), trained the model on thousands of jokes, looked at how the loss changed during training, plotted the graph of how they changed per epoch, and generated new jokes to see how well the model learned what I trained it on.

In the second part, I built three different models to understand how memes can be classified using images, text, or both together. I created a CNN to classify memes using only images, an RNN to classify them using only text, and a multimodal model that combines both image and text features. I tested all three models using accuracy and F1 scores to see how well each one performed.

The purpose of this assignment is to compare single-modality models (image-only or text-only) with a multimodal model and see which one works best. By doing these experiments, I was able to observe how combining both text and images can significantly help improve the performance of a classification model. As expected, the model together did show some changes while observing.

1. Language Modelling(q1_1, q1_2, q1_3)

A. Language model on joke dataset using Transformer Decoder architecture:

To start this task, I first installed the required Python packages. I used Anaconda to create and manage my environment, and installed the necessary libraries (such as torch, tokenizers, and transformers) using both pip and conda. This arrangement setup was important because training a language model from the start requires several libraries that are not included in Python by default. For the

language modeling part, I train a Byte-Pair Encoding (BPE) tokenizer from the start. This performs all the work to manage and fragment the jokes into small units, which helps the model handle rare, difficult, or different words and humor-specific vocabulary.

After the tokenizer is built, I created a GPT-2–style decoder-only Transformer model with the following configuration: 4 layers, 4 attention heads, 256-dimensional embeddings, 128-token context window

Then the model was trained for 1 epoch in the beginning to make sure it was working on the full 231k jokes dataset. It took more than 10 hours to do so.



Fig1.1: Model training successful with 1 epoch.

This part of the assignment helped me understand how language models learn from text, how tokenizers impact the model's behavior or performance, and what training from start to end looks like compared to using already trained and developed models.

B. Train the joke language model for 10 epochs:

For this part training in 10 epoch, I trained the language model for 10 epochs to see changes better or nor it could learn the joke dataset better or not, compared to the 1-epoch version.

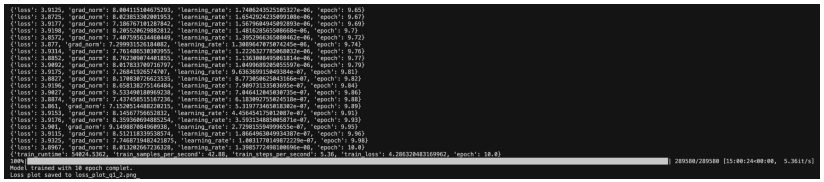


Fig. 1.2: The Model trained on 10 epochs was successful.

The loss curve in the graph displays a clear learning pattern of changes: the model starts with a very high loss of about 7.45, which states that it initially struggles to predict the next token in an organized manner correctly. The loss drops quickly after the first epoch, showing that the model is quickly learning basic patterns from the jokes. As training goes on, the loss gradually decreases at a slower pace and constant rate. At epoch 4, around that point, the curve starts to flatten,

and by epoch 10, the loss becomes stable around 4-6. This tells us the model is keeping the learning, but at a slow rate, and it has to get to a point where improvements become smaller, which is not harder to get around or detect. In the conclusion, the graph displays that training for many epochs helps the model become more stable, consistency and better at understanding joke format and meaning, even when running on a CPU-only environment used locally.

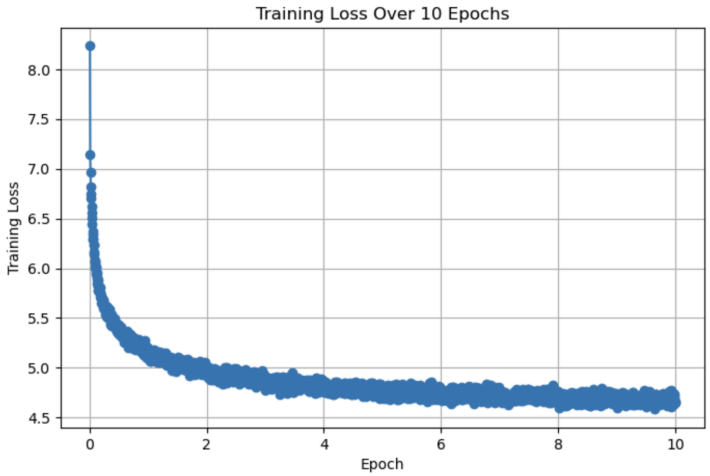


Fig1.3Training graph

{'train_runtime': 24803.0621, 'train_samples_per_second': 93.134, 'train_steps_per_second': 46.567, 'train_loss': 4.866115677632914, 'epoch': 10.0}

Hardware and Environment

- Apple MacBook Pro (Apple M3)
- CPU-only training
- 16 GB RAM
- Python environment managed with Anaconda
- Libraries installed via pip and conda
- Training time: ~about 5 hours

C. Trained model to generate at least 5 new jokes:

For this part of the assignment, I trained the Transformer language model to generate five new jokes. I loaded my custom tokenizer and the GPT-2–style model from Q1.2, and used the text-generation pipeline to explain the simple process. I provided different prompts such as “Why did the chicken”, “my dog”, “what do you call”, “my friend told me”, and “knock knock” to create a joke. The model then displays the short joke-like completions for each prompt as an output. To generate jokes, reading and reality, I limited the model’s output using max_new_tokens=25 in the text-generation pipeline.

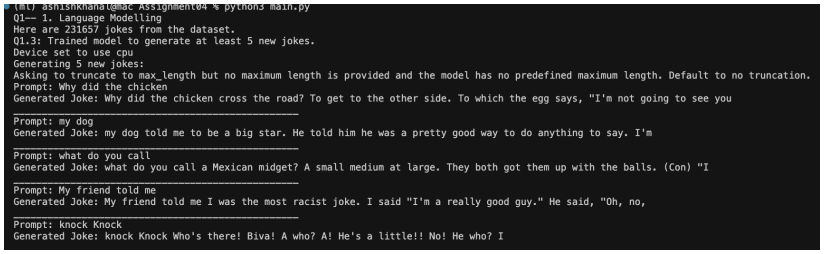


Fig. 1.4 Five jokes from the trained mode.

D. Qualitative evaluation:

Finally, for the evaluation part, I use a trained Transformer of a 10-epoch language model to generate five new jokes using the prompts below.

```
prompt = ["Why did the chicken", "who is under", "what
do you call", "I have to go", "knock Knock"]
```

The output jokes were a mix of humorous, unexpected, and sometimes random, not making any sense, which is as expected in a Transformer trained from the beginning. I set *max_new_tokens = 25* to keep each output logical and readable. Prompted with “Why did the chicken”, the model produced a punchline about crossing the road. With “my dog”, it generated a meaningful sentence about the dog being a “big star.” Some outputs also display the model repeating or frequency into a non-logical idea, which means the limitations of training on a CPU with the resources we have available.

Overall, this part helps to understand that the model was able to learn joke patterns from the dataset and produce creative text, which, with better training it can get better. The model was able to find some elements of humor and natural language flow sense while still showing clear limitations in coherence, joke quality, and more.

2. Multimodal Learning

A. Convolutional Neural Network (CNN) using PyTorch

For this task of CNN, I first split the meme image dataset into 60% training, 20% validation, and 20% testing. I used the sklearn library for this, following the assignment instructions. Using the labels.xlsx file, I mapped each image to its label and created three subsets: displayed in the figure, 4194 training images, 1398 validation images, and 1399 test images. I then made a custom PyTorch Convolutional Neural Network named MyCNN, as mentioned in the question, which has four convolutional layers and four pooling layers, as required. All images were resized to 128×128, converted to RGB, and transformed into tensors. Since one image file was missing, it

is creating a problem in the dataset. I added a function that replaces missing images with a blank placeholder to get around training errors. The model was trained at 10 epochs using the Adam optimizer and cross-entropy loss.

After training and completion up to 10 epochs, I examined MyCNN on the test set and generated the accuracy and F1 metrics. The results were as mentioned in Figure 1.5: Micro Accuracy = 0.5947, Macro Accuracy = 0.3333, Micro F1 = 0.5947, and Macro F1 = 0.2486. These outputs show that the model performs reasonably well on most of the classes but struggles with minority classes, as expected due to class not being equal in the dataset. Overall, MyCNN learns general visual patterns but has a limited performance to completely understand meme meaning using images alone. This sets the key objective for later parts of the assignment, where text-only and multimodal models are used that help improve classification performance, but not in all cases.

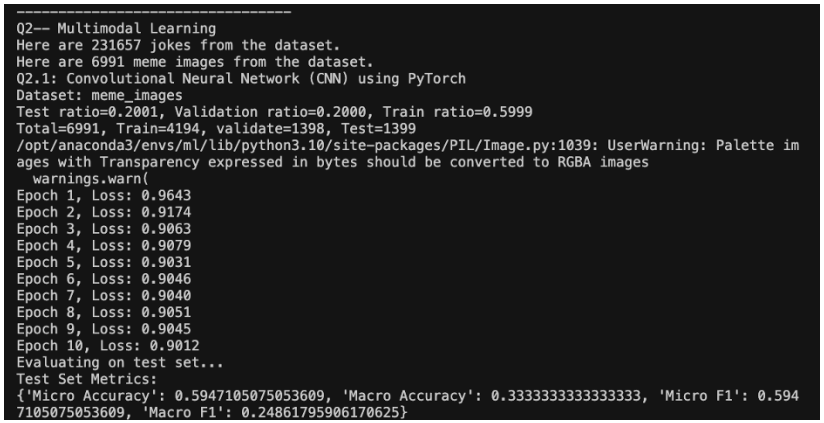


Fig. 1.5 (CNN) using PyTorch with 10 epochs.

B. Recurrent Neural Network(RNN) using PyTorch

For this part of the assignment using the ANN, I start with the meme text dataset and split it into 60% training, 20% validation, and 20% testing, using stratified sampling to preserve the label distribution. Here, I use the sklearn library techniques. Then I designed an RNN model in PyTorch called MyRNN as given in the question, which has an embedding layer, a single-layer RNN, and a fully connected classification layer. At the beginning of the training, I tokenized the meme text using(*basic_english*)and built a vocabulary from the training set, which is a lot easier. Then all of the sequences are using a custom collate function to ensure same stable batch

number and sizes. The model was then trained for 10 epochs, using cross-entropy loss and the Adam optimizer same as we did in the CNN model.

After training for a few minutes faster than CNN, I observed that MyRNN on the test set and calculated all required metrics: Micro Accuracy, Macro Accuracy, Micro F1 score, and Macro F1 score. The results were as shown in Figure 1.6: Micro Accuracy: 0.5933, Macro Accuracy: 0.3336, Micro F1: 0.5933, Macro F1: 0.2527. These results show that the RNN performs reasonably well in most of the classes, as in the micro scores sections, but struggles with minority classes, resulting in lower macro accuracy and macro F1. This means that the dataset is not properly balanced, and the simple RNN design may not detect enough semantic information from meme text. Overall, in the conclusion, this model learned meaningful patterns but could benefit from improvements that were made.

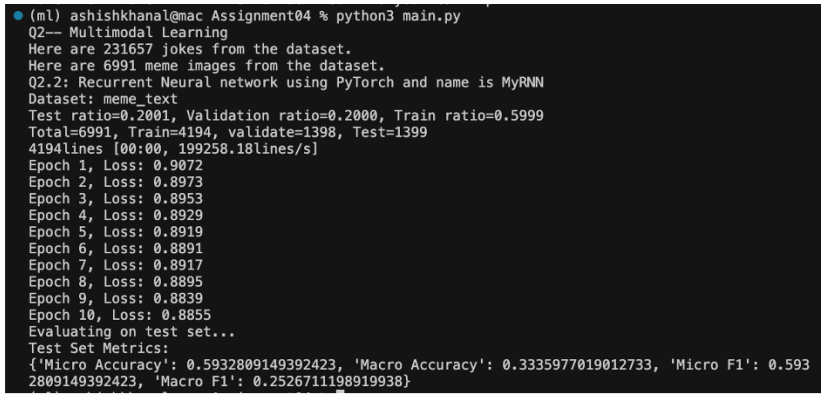


Fig. 1.6 (ANN) using PyTorch with 10 epochs.

C. PyTorch uses MyMultimodalModel, using MyCNN and MyRNN

For this task, I built a multimodal model, MyMultimodalModel name in the question, by combining the CNN from Q2.1 and the RNN from Q2.2 altogether. At first, same as for the above section, I split the meme dataset into training, validation, and test sets using the required 60/20/20 ratio using the sklearn methods. The CNN (MyCNN) changes and translates each meme image into a 128-dimensional feature vector, and the RNN (MyRNN) translates the meme text into another 128-dimensional vector. I then used both vectors in a single 256-dimensional representation and passed them through fully connected layers for multi-class sentiment classification.

The dataset includes 6,991 samples, and I trained the multimodal model for 10 epochs using cross-entropy loss and Adam optimization, as can also be seen in the image below. After training, I examined the model on the test set and obtained the following metrics, as shown in Figure 1.7: Micro Accuracy: 0.5918, Macro Accuracy: 0.3324, Micro F1: 0.5918, Macro F1: 0.2510

These results show that the multimodal model performs same as the image-only and text-only models in terms of micro accuracy, but the macro accuracy and macro F1 remain low, which is something that is unexpected, but it happened here. This means that the model still struggles with small classes and unequal label management. While combining image and text features should automatically improve performance but in our case, this didn't happen. The gains here were just fractions, not much hard to evaluate. Reasons this might have happened include: weak text quality, limited CNN/RNN capacity, missing or low-quality images, and the simple RNN architecture used. Split or the SEED used might also affect this, but moreover over it's not a normal result as we expect from the combined dataset. Despite this, the multimodal architecture successfully combines two different feature types and shows how multimodal learning works in real life.

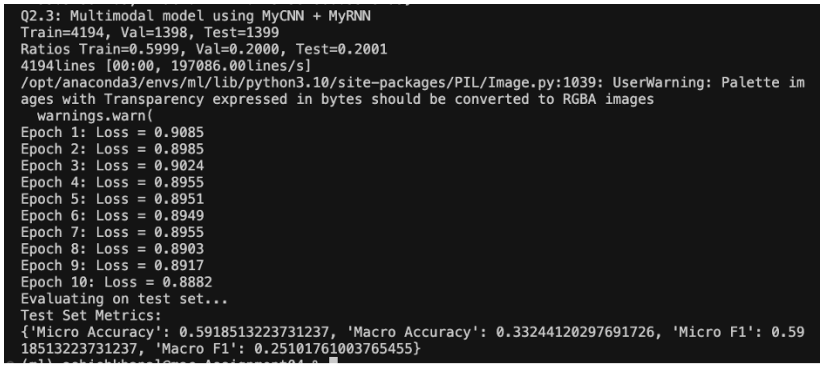


Fig. 1.7 CNN and ANN using PyTorch with 10 epochs